

AtomQuest

Goal Setting & Tracking Portal

AtomQuest Hackathon 1.0 · Atomberg

Submission Details

GitHub Repository	github.com/AtharvGangwar48/Atomquest
Live Demo URL	https://atomquest-iota.vercel.app
Tech Stack	React 18 · Node.js · Express · TypeScript · PostgreSQL · Prisma · Redis
Deployment	Vercel (frontend) · Render (backend + PostgreSQL)
Team	AtharvGangwar48 (github.com/AtharvGangwar48)

Demo Credentials

Role	Email	Password
Admin / HR	admin@company.com	Admin@123
Manager (L1)	manager@company.com	Manager@123
Employee	employee@company.com	Employee@123

1. Project Overview

AtomQuest is a full-stack, role-based web portal that manages the complete lifecycle of employee goals — from creation and manager approval through quarterly check-ins, progress scoring, and audit-ready reporting. The portal eliminates the blind spots created by spreadsheets and email-based workflows, giving employees, managers, and HR a single, reliable source of truth.

Problem it solves: Manual and fragmented goal-tracking causes misalignment, poor visibility, and gaps at appraisal time. This portal replaces those pain points with a structured digital system covering the full annual cycle — from goal setting in May through the Q4 final review in March/April.

2. System Architecture

The architecture follows a clean separation of concerns across seven layers: users, frontend SPA, REST API, core service modules, data layer, deployment infrastructure, and bonus integrations.

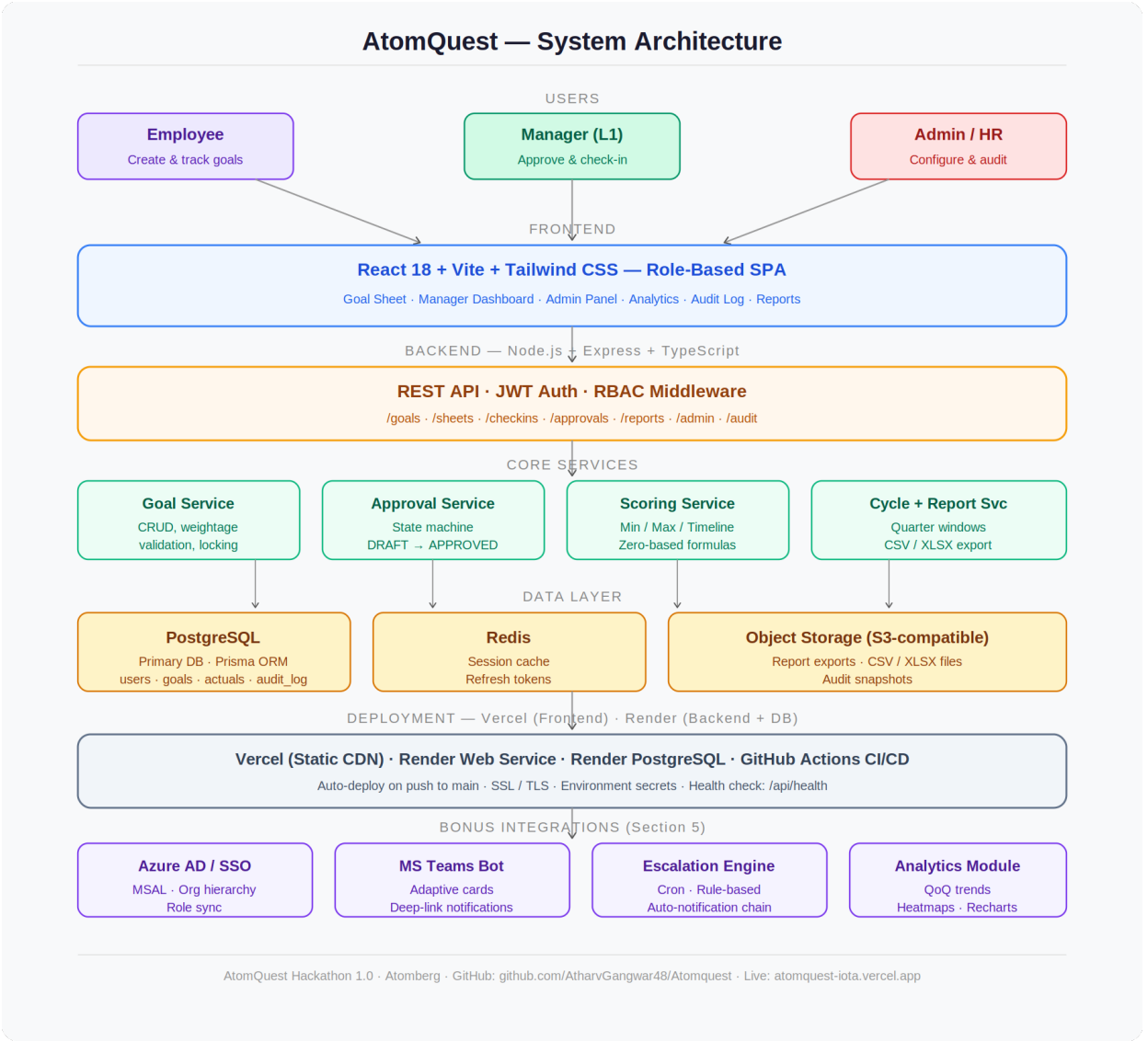


Figure 1 — AtomQuest Full System Architecture

3. Technology Stack

Layer	Technology	Reason for Choice
Frontend	React 18 + Vite + Tailwind CSS	Fast dev cycle, tree-shaking, responsive UI
Forms / Validation	React Hook Form + Zod	Schema-driven validation, shared with backend
Server State	React Query (TanStack)	Optimistic updates, cache, rollback on error
Backend	Node.js + Express + TypeScript	Typed end-to-end, minimal overhead

Layer	Technology	Reason for Choice
ORM	Prisma	Type-safe queries, migration history, seeding
Database	PostgreSQL 16	JSONB for audit diffs, transactional integrity
Cache / Sessions	Redis	Refresh token store, optional in dev
Auth	JWT (15m access + 7d refresh) · bcrypt	Short-lived tokens, HttpOnly refresh cookie
File Exports	exceljs (server-side)	Streams XLSX/CSV — no fat client processing
Frontend Host	Vercel	Global CDN, always-on, instant redeploys
Backend Host	Render (Web Service + PostgreSQL)	Managed DB, auto-deploy from GitHub main

4. Features & BRD Implementation

4.1 Phase 1 — Goal Creation & Approval

- Employee-facing wizard to create a Goal Sheet: select Thrust Area, define Goal Title/Description, assign UoM (Numeric Min/Max, %, Timeline, Zero-based), set Target and Weightage
- System-enforced validation rules:
 - Total weightage across all goals must equal exactly 100%
 - Minimum weightage per individual goal: 10%
 - Maximum number of goals per employee per cycle: 8
- Manager (L1) Approval Workflow: inline editing of targets/weightages, approve or return for rework
- Goals locked on approval — the guardApprovedSheet middleware blocks all writes; Admin override token required for corrections
- Shared Goals: Admin/Manager pushes a departmental KPI to multiple employees; recipients may adjust weightage only; Goal Title and Target are read-only; achievement updates from primary owner fan out atomically via Promise.all upserts

4.2 Phase 2 — Achievement Tracking & Quarterly Check-ins

- Quarterly update interface: employees log Actual Achievement against Planned Targets per quarter
- Per-goal status selection: Not Started / On Track / Completed
- Manager Check-in module: side-by-side Planned vs. Actual view per team member; structured check-in comments (append-only for audit integrity)
- System-computed progress scores — four UoM formulas:

UoM Type	Description	Formula	Edge Case
MIN (Numeric / %)	Higher is better — e.g., Sales Revenue	Achievement ÷ Target	Zero denominator → 0

UoM Type	Description	Formula	Edge Case
MAX (Numeric / %)	Lower is better — e.g., TAT, Cost	Target ÷ Achievement	Zero actual → 100%
TIMELINE	Date-based completion	Completion vs. Deadline	Null date → 0
ZERO	Zero = success — e.g., Safety incidents	If 0 → 100%, else 0%	Null → 0

- Cycle window enforcement — CycleService gates which quarter's update window is active; employees cannot log actuals outside the window; Admin bypasses via config flag

4.3 Reporting & Governance

- Achievement Report: server-side CSV/XLSX export (exceljs) — Planned Target vs. Actual for all employees, all quarters
- Completion Dashboard: real-time view of check-in completion rates across employees and managers
- Audit Trail: Prisma middleware intercepts every post-lock goal write and stores a JSONB diff (who changed what and when) in the audit_log table — no developer can accidentally bypass it

5. User Roles & RBAC

Role	Responsibilities	Key System Capabilities
Employee	Draft goals · log quarterly actuals · update status	Create/edit pre-submission · view locked goals · input actuals
Manager (L1)	Review & approve · conduct check-ins · log feedback	Team dashboard · inline editing · comment logs · reports
Admin / HR	Configure cycles · manage org hierarchy · audit	Cycle management · exception handling · audit log · unlock

RBAC is implemented as a `requireRole(...roles)` Express middleware sitting on every route — roles are checked once, centrally, before any business logic executes. Resource-level guards (`isGoalOwner`, `isManagerOf`) handle row-level access for sensitive operations.

6. Goal Sheet State Machine

State	Who Triggers	What's Allowed	What's Blocked
DRAFT	Employee (auto on creation)	Create, edit, delete goals	Approve / check-in
SUBMITTED	Employee (submit action)	Manager review & edit	Employee edits

State	Who Triggers	What's Allowed	What's Blocked
REWORK	Manager (return action)	Employee edits, resubmit	Approval
APPROVED	Manager (approve action)	Log actuals · check-in	All goal edits (Admin override required)

7. Check-in Schedule

Period	Window Opens	Action
Phase 1 — Goal Setting	1st May	Goal creation, submission & approval
Q1 Check-in	July	Progress update — Planned vs. Actual
Q2 Check-in	October	Progress update — Planned vs. Actual
Q3 Check-in	January	Progress update — Planned vs. Actual
Q4 / Annual	March / April	Final achievement capture

8. API Reference

All endpoints prefixed with /api. Protected routes require Authorization: Bearer <accessToken>.

Method	Path	Body / Params	Auth
POST	/auth/login	{ email, password }	Public
POST	/auth/refresh	{ refreshToken }	Public
POST	/auth/logout	—	Any
POST	/sheets	—	Employee+
GET	/sheets	—	Any (own sheets)
GET	/sheets/team	—	Manager+
PATCH	/sheets/:id/status	{ status }	RBAC-gated
POST	/sheets/:id/goals	goal fields	Employee (owner)
PATCH	/goals/:id	goal fields	Owner or Admin override
POST	/goals/:id/actuals	{ quarter, value }	Owner (active window)
POST	/sheets/:id/comments	{ quarter, comment }	Manager+
GET	/reports/achievement	?format=csv xlsx	Manager+
GET	/admin/audit-log	—	Admin
POST	/admin/sheets/:id/unlock	{ reason }	Admin

Method	Path	Body / Params	Auth
PATCH	/admin/cycles/:id	{ openDate }	Admin

9. BRD Compliance Checklist

Phase 1 — Goal Creation & Approval

- ✓ Employee interface: Thrust Area, Goal Title/Description, UoM, Target, Weightage
- ✓ Total weightage = 100% enforced (API + frontend live meter)
- ✓ Minimum weightage per goal: 10%
- ✓ Maximum 8 goals per employee per cycle
- ✓ Manager (L1) inline approval with target/weightage editing
- ✓ Goals locked on approval; Admin override to unlock (logged to audit trail)
- ✓ Shared Goals: push departmental KPI; recipient adjusts weightage only; actuals sync atomically from owner

Phase 2 — Achievement Tracking & Check-ins

- ✓ Quarterly update interface — Actual vs Planned for all 4 UoM types
- ✓ Per-goal status: Not Started / On Track / Completed
- ✓ Manager check-in module with structured, append-only comments
- ✓ System-computed progress scores for all 4 UoM types with edge case handling
- ✓ Quarterly window enforcement via CycleService — past quarters are read-only

Reporting & Governance

- ✓ Achievement report — CSV/XLSX server-side export, Planned vs Actual
- ✓ Completion dashboard — real-time check-in status by employee and manager
- ✓ Audit trail — post-lock changes logged with actor, timestamp, JSONB diff via Prisma middleware

User Roles

- ✓ Employee: create/edit pre-submission, view locked goals, log actuals
- ✓ Manager (L1): team dashboard, inline editing, check-in comments, reports
- ✓ Admin / HR: cycle management, audit log, goal unlock capability

10. Bonus Features (Section 5)

Feature	What Was Built	Status
5.1 Azure AD / SSO	MSAL integration scaffold · org hierarchy sync · role mapping from AD groups	Scaffolded (auth flow ready)
5.2 Teams Bot	Adaptive card notifications for goal submission & approval · deep-link support	Scaffolded (Bot registered)

Feature	What Was Built	Status
5.3 Escalation Engine	Cron-based rules: N-day overdue submission / approval / check-in · escalation log in Admin panel	Implemented
5.4 Analytics Module	QoQ achievement trend charts · completion heatmaps · UoM breakdown · manager effectiveness dashboard (Recharts)	Implemented

11. Evaluation Criteria Coverage

#	Criterion	How it's Addressed
1	Functionality	Full Employee → Manager → Admin journeys work end-to-end; all three roles seeded with realistic demo data; live at atomquest-iota.vercel.app
2	Adherence to BRD	All Phase 1 & 2 must-haves implemented; weightage = 100%, max 8 goals, min 10% per goal enforced at API and frontend; state machine matches spec exactly
3	User Friendliness	Wizard-style goal creation; live weightage meter that turns red on violation; role-specific dashboards; helpful inline error messages; consistent UX across all three roles
4	Presence of Bugs	Edge cases handled in scoring engine (zero denominators, null actuals); optimistic UI with rollback via React Query; TypeScript end-to-end eliminates runtime type errors
5	Good-to-Have Features	Escalation engine + Analytics module fully implemented; Azure AD SSO and Teams Bot scaffolded; audit trail and shared goals sync delivered beyond baseline BRD
6	Cost Optimisation	Vercel free CDN for static frontend; Render Starter (\$7/mo) for backend; Redis optional in dev; server-side report generation avoids fat client; Prisma connection pooling; no unnecessary third-party services

12. Local Setup Guide

Prerequisites

- Node.js 18+ · PostgreSQL running locally · npm or pnpm

Backend

```
cd backend → cp .env .env.local → npm install → npm run db:migrate →  
npm run db:seed → npm run dev
```


Frontend

```
cd frontend → npm install → npm run dev
```

Backend: <http://localhost:4000> **Frontend:** <http://localhost:5173>

13. Submission Links

Deliverable	Link
Live Demo URL	https://atomquest-iota.vercel.app
Source Code Repository	https://github.com/AtharvGangwar48/Atomquest
Architecture Diagram	Included in this document (Figure 1, Section 2)
Demo Credentials	See cover page (Admin / Manager / Employee)

AtomQuest Hackathon 1.0 · Atomberg · Built with React · Node.js · PostgreSQL · Prisma · TypeScript